LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH KOTLIN

Oleh:

Muhammad Nafis Putra NIM. 2410817210021

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Kotlin ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Nafis Putra
NIM : 2410817210021

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Raymond Hariyono	Erika Maulidiya, S.Kom., M.Kom.
NIM. 2310817210007	NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 9
Source Code XML 9
Source Code Compose 13
B. Output Program 17
Output XML 17
Output Compose 18
C. Pembahasan 19
Pembahasan XML 19
Pembahasan Compose 22
SOAL 2 25
SOAL 3 26
Daftar Pustaka 27
Tautan Git 28

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 XML..... 17
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose 17
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose 18
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose 18

DAFTAR TABEL

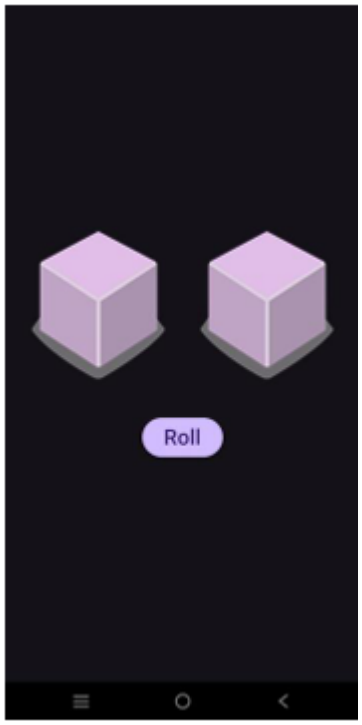
Tabel 1. Source Code Jawaban Soal 1 XML 9
Tabel 2. Source Code Jawaban Soal 1 XML 11
Tabel 3. Source Code Jawaban Soal 1 Compose 13

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.

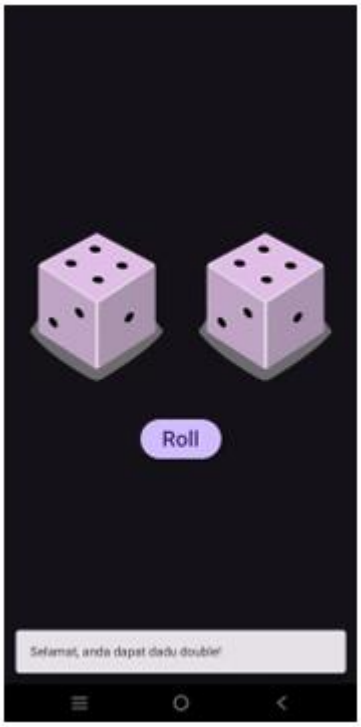


Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.

Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double” seperti dapat dilihat pada Gambar 3.



Gambar 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam folder Modul 1 dalam bentuk Project. Jangan lupa untuk melakukan Clean Project sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

Source Code XML

MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 XML

```
package com.example.dice

import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private var currentToast: Toast? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rollButton: Button = findViewById(R.id.btnRoll)

        rollButton.setOnClickListener {
            rollDice()
        }
    }

    private fun rollDice() {
        val dice1 = Dice(6)
        val diceRoll1 = dice1.roll()

        val dice2 = Dice(6)
        val diceRoll2 = dice2.roll()

```

```
        val diceImage1: ImageView = findViewById(R.id.ivDice1)
        val diceImage2: ImageView = findViewById(R.id.ivDice2)

        val drawableResource1 = when (diceRoll1) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }

        val drawableResource2 = when (diceRoll2) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }

        diceImage1.setImageResource(drawableResource1)
        diceImage1.contentDescription = diceRoll1.toString()

        diceImage2.setImageResource(drawableResource2)
        diceImage2.contentDescription = diceRoll2.toString()

        currentToast?.cancel()

        if (diceRoll1 == diceRoll2) {
            currentToast = Toast.makeText(this, "Selamat, anda dapat dadu double", Toast.LENGTH_SHORT)
```

```
        } else {
            currentToast = Toast.makeText(this, "Anda belum beruntung!", Toast.LENGTH_SHORT)
        }

        currentToast?.show()
    }
}

class Dice(private val numSides: Int) {
    fun roll(): Int {
        return (1..numSides).random()
    }
}
```

activity_main.xml

Tabel 2. Source Code Jawaban Soal 1 XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:background="@android:color/background_light">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center">

        <ImageView
```

```
            android:id="@+id/ivDice1"
            android:layout_width="170dp"
            android:layout_height="170dp"
            android:src="@drawable/dice_0" />

        <Space
            android:layout_width="16dp"
            android:layout_height="wrap_content" />

        <ImageView
            android:id="@+id/ivDice2"
            android:layout_width="170dp"
            android:layout_height="170dp"
            android:src="@drawable/dice_0" />
    </LinearLayout>

    <Space
        android:layout_width="wrap_content"
        android:layout_height="32dp" />

    <Button
        android:id="@+id/btnRoll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Roll" />

</LinearLayout>
```

Source Code Compose

MainActivity.kt

Tabel 3. Source Code Jawaban Soal 1 Compose

```
package com.example.dice

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.wrapContentSize
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.tooling.preview.Preview
```

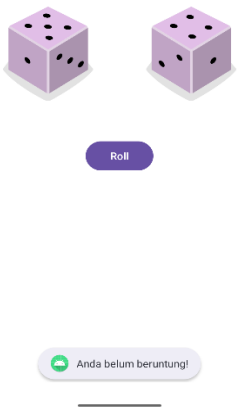
```
import androidx.compose.ui.unit.dp
import com.example.dice.ui.theme.DiceTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            DiceTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color                                    =
MaterialTheme.colorScheme.background
                ) {
                    DiceRollerApp()
                }
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun DiceRollerApp() {
    DiceWithButtonAndImages(
        modifier = Modifier
            .fillMaxSize()
            .wrapContentSize(Alignment.Center)
    )
}

@Composable
fun DiceWithButtonAndImages(modifier: Modifier = Modifier) {
    var result1 by remember { mutableIntStateOf(0) }
```

```
    var result2 by remember { mutableIntStateOf(0) }

    val context = LocalContext.current

    val imageResource1 = when (result1) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    val imageResource2 = when (result2) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Row(
            horizontalArrangement = Arrangement.Center,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Image(
```

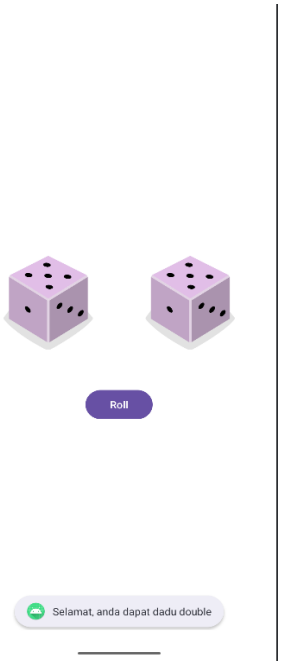
```
                painter = painterResource(id = imageResource1),
                contentDescription = result1.toString()
            )

            Spacer(modifier = Modifier.width(16.dp))

            Image(
                painter = painterResource(id = imageResource2),
                contentDescription = result2.toString()
            )
        }

        Spacer(modifier = Modifier.height(32.dp))

        Button(onClick = {
            result1 = (1..6).random()
            result2 = (1..6).random()

            if (result1 == result2) {
                Toast.makeText(context, "Selamat, anda dapat dadu double", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(context, "Anda belum beruntung!", Toast.LENGTH_SHORT).show()
            }
        }) {
            Text(text = "Roll")
        }
    }
}
```

B. Output Program

Output XML

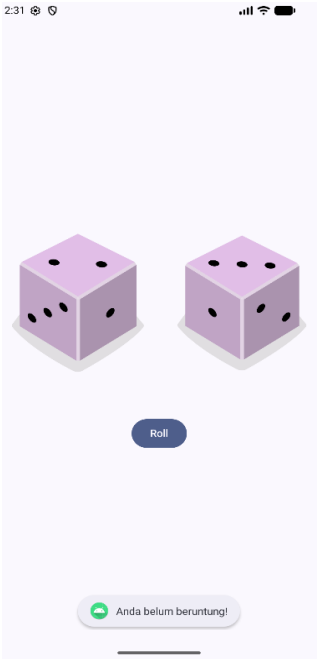


Gambar 1. Screenshot Hasil Jawaban Soal 1 XML

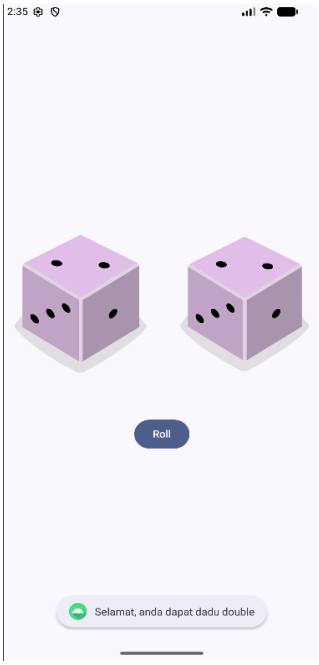


Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose

Output Compose



Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

Pembahasan XML

MainActivity.kt:

Struktur kode aplikasi dice menggunakan XML dengan LinearLayout sebagai kontainer utama dengan atribut android:orientation="vertical" pada baris ke-6 untuk menempatkan unsur secara tegak lurus. Di dalam kontainer tersebut, terdapat LinearLayout pada baris ke-9 dengan horizontal yang berfungsi sebagai tempat untuk menyejajarkan dua komponen ImageView. Komponen pertama dengan ID ivDice1 pada baris ke-15 dan komponen kedua dengan ID ivDice2 pada baris ke-26 masing-masing memiliki dimensi lebar dan tinggi sebesar 170dp untuk menampilkan visualisasi dadu. Sebagai unsur hubungan, sebuah Button didefinisikan pada baris ke-40 dengan ID btnRoll yang diletakkan di bawah susunan gambar dengan pemisah berupa Space setinggi 32dp pada baris ke-36 untuk menjaga kerapian tata letak.

Pada bagian logika program MainActivity.kt, inisialisasi aktivitas dimulai dengan pemanggilan setContentView(R.layout.activity_main) pada baris ke-15 yang menghubungkan kode Kotlin dengan desain XML yang telah dibuat. Variabel rollButton pada baris ke-17 dideklarasikan untuk memegang referensi tombol melalui metode findViewById(R.id.btnRoll) sehingga saat klik dapat dideteksi. Instruksi rollButton.setOnClickListener pada baris ke-19 menetapkan sebuah blok kode yang akan dieksekusi setiap kali tombol ditekan, di mana di dalamnya terdapat pemanggilan fungsi rollDice(). Selain itu, terdapat deklarasi variabel currentToast pada baris ke-11 yang bersifat *nullable* untuk mengelola tampilan pesan singkat agar tidak terjadi penumpukan notifikasi pada layar saat tombol ditekan berkali-kali secara cepat.

Proses pengacakan angka dilakukan melalui penggunaan kelas Dice yang didefinisikan pada baris ke-71 dengan parameter numSides untuk menentukan jumlah sisi dadu. Fungsi roll() pada baris ke-72 mengembalikan nilai integer acak menggunakan jarak (1..numSides).random() yang mensimulasikan lemparan dadu secara nyata. Di dalam fungsi rollDice(), dua objek dari kelas Dice diinstansiasi pada baris ke-26 dan ke-29 untuk mendapatkan dua angka acak yang berbeda bagi masing-masing ImageView. Hasil dari

pengacakan ini kemudian disimpan ke dalam variabel diceRoll1 dan diceRoll2 untuk diproses lebih lanjut ke tahap pemilihan aset gambar yang sesuai.

Menampilkan visualisasi gambar dilakukan melalui struktur kontrol when yang dimulai pada baris ke-34 untuk dadu pertama dan baris ke-43 untuk dadu kedua. Logika ini memetakan angka hasil acakan ke dalam referensi *resource* gambar, seperti R.drawable.dice_1 untuk angka satu hingga R.drawable.dice_6 untuk angka enam yang ditangani oleh cabang else pada baris ke-40 dan ke-49. Setelah referensi gambar didapatkan dan disimpan dalam variabel drawableResource, instruksi setImageResource() pada baris ke-53 dan ke-56 secara otomatis memperbarui tampilan ivDice1 dan ivDice2 pada layar. Pemberian nilai pada contentDescription di baris ke-54 dan ke-57 juga dilakukan untuk mendukung fitur aksesibilitas dengan mengonversi angka hasil kocokan menjadi string.

Proses antarmuka pada bagian akhir fungsi rollDice() melibatkan penggunaan currentToast?.cancel() di baris ke-59 untuk menghentikan pesan Toast yang mungkin masih muncul dari aksi sebelumnya. Langkah ini sangat teknis untuk memastikan bahwa pengguna mendapatkan umpan balik yang instan tanpa harus menunggu durasi pesan lama berakhir. Kondisi percabangan if (diceRoll1 == diceRoll2) pada baris ke-61 mengevaluasi apakah kedua angka yang keluar memiliki nilai yang sama untuk menentukan jenis pesan yang akan ditampilkan. Jika kondisi tersebut terpenuhi, objek Toast diisi dengan teks "Selamat, anda dapat dadu double" pada baris ke-62, sementara kondisi sebaliknya akan pesan "Anda belum beruntung!" pada baris ke-64.

Struktur kode tersebut ditutup dengan instruksi currentToast?.show() pada baris ke-67 yang munculnya kotak dialog pesan di bagian bawah layar perangkat. Secara keseluruhan, keterkaitan antara XML dan MainActivity.kt berfokus pada sinkronisasi ID komponen agar setiap perubahan data pada logika pemrograman dapat langsung tercermin pada unsur visual. Penggunaan kelas terpisah untuk logika pengacakan menunjukkan penerapan prinsip enkapsulasi sederhana agar kode Implementasi antarmuka pada kode ini menggunakan Jetpack Compose, di mana seluruh unsur UI didefinisikan secara deklaratif di dalam fungsi-fungsi Composable. Pada MainActivity baris ke-35, instruksi setContent digunakan untuk menetapkan tata letak aktivitas dengan membungkus DiceRollerApp di dalam DiceTheme dan Surface. Penggunaan Modifier.fillMaxSize() pada baris ke-40 memastikan bahwa

aplikasi mengisi seluruh ruang layar yang tersedia dengan latar belakang yang mengikuti skema warna tema yang telah ditentukan.

Fungsi DiceRollerApp pada baris ke-51 berfungsi sebagai entri utama untuk pratinjau dan pemanggilan komponen inti. Di dalamnya, DiceWithButtonAndImages dipanggil dengan modifier wrapContentSize(Alignment.Center) pada baris ke-55, yang secara teknis memusatkan seluruh konten di tengah layar. Struktur utama di dalam komponen ini menggunakan Column pada baris ke-85 untuk mengatur unsur secara vertikal dengan penyelarasan horizontal di tengah melalui properti Alignment.CenterHorizontally.

Pembuatan status angka dadu dilakukan menggunakan variabel result1 dan result2 pada baris ke-60 dan ke-61 dengan mamanfaatkan remember { mutableIntStateOf(0) }. Penggunaan remember sangat krusial dalam Jetpack Compose untuk menjaga nilai variabel agar tidak hilang saat terjadi proses *recomposition* atau penggambaran ulang antarmuka. Nilai awal disetel ke angka nol, yang nantinya akan perubahan sumber daya gambar saat tombol ditekan dan nilai variabel tersebut diperbarui.

Visualisasi dadu disusun di dalam sebuah Row pada baris ke-89 yang memungkinkan dua buah komponen Image diletakkan secara berdampingan secara horizontal. Penentuan gambar yang ditampilkan dikelola oleh variabel imageResource1 pada baris ke-65 dan imageResource2 pada baris ke-75 melalui logika percabangan when. Setiap angka hasil acakan dipetakan ke aset gambar yang sesuai di dalam folder *drawable*, seperti R.drawable.dice_1 hingga R.drawable.dice_6, yang kemudian berada ke layar menggunakan fungsi painterResource pada baris ke-94 dan ke-101.

Hubungan utama ada dalam komponen Button pada baris ke-108 yang memiliki blok onClick untuk mengeksekusi logika pengocokan dadu. Baris ke-109 dan ke-110 menjalankan fungsi (1..6).random() untuk memperbarui nilai result1 dan result2 secara acak. Karena variabel tersebut merupakan *state*, perubahan nilai akan secara otomatis UI untuk memperbarui tampilan gambar secara *real-time* tanpa perlu memanggil fungsi pembaruan secara manual seperti pada sistem View konvensional.

Logika pengecekan kondisi untuk memunculkan pesan dilakukan pada baris ke-112 dengan membandingkan kesamaan nilai antara kedua dadu. Fungsi LocalContext.current

pada baris ke-63 diperlukan untuk mendapatkan konteks aplikasi agar Toast.makeText dapat dipanggil di dalam fungsi Composable. Jika kedua dadu menghasilkan angka yang sama, pesan keberhasilan akan muncul melalui baris ke-113, sementara jika tidak sama, pesan pada baris ke-115 akan ditampilkan untuk memberikan umpan balik langsung kepada pengguna mengenai hasil kocokan tersebut.

tetap tersusun dan mudah didevelop. Mekanisme ini memastikan bahwa setiap hubungan tombol menghasilkan alur lengkap mulai dari pengacakan data, pembaruan sumber daya gambar, hingga pemberian notifikasi status kepada pengguna secara real-time.

Pembahasan Compose

MainActivity.kt:

Kode diawali dengan package com.example.dice pada baris ke-1 yang berfungsi sebagai penentu struktur folder proyek aplikasi. Selanjutnya, dari baris ke-3 hingga ke-32 berisi kumpulan import untuk memanggil berbagai pustaka bawaan Android dan Jetpack Compose yang diperlukan agar komponen antarmuka dapat berjalan. Kelas utama aplikasi didefinisikan pada baris ke-34 dengan kode class MainActivity: ComponentActivity(). Di dalam kelas ini, fungsi override fun onCreate(savedInstanceState: Bundle?) pada baris ke-35 menjadi titik awal eksekusi program. Pada baris ke-37, perintah enableEdgeToEdge() dieksekusi agar tampilan aplikasi dapat membentang memenuhi seluruh batas layar perangkat. Antarmuka aplikasi kemudian diatur mulai dari baris ke-38 menggunakan fungsi setContent, di mana komponen Surface pada baris ke-40 diterapkan dengan modifikasi modifier = Modifier.fillMaxSize() pada baris ke-41 untuk memastikan area kanvas menutupi ruang layar secara penuh dan memuat komponen DiceRollerApp() pada baris ke-44.

UI pembungkus utama dibuat dalam sebuah fungsi bernama DiceRollerApp() pada baris ke-53. Fungsi ini ditandai dengan anotasi @Preview(showBackground = true) pada baris ke-51 yang berguna untuk memunculkan pratinjau desain secara langsung di area pengembangan, serta anotasi @Composable pada baris ke-52 agar dikenali sebagai pembentuk tampilan visual Jetpack Compose. Di dalam fungsi ini, komponen DiceWithButtonAndImages dipanggil pada baris ke-54. Pemanggilan tersebut menyertakan pengaturan tata letak melalui baris ke-56 dengan kode .fillMaxSize() dan baris ke-57 dengan

kode .wrapContentSize(Alignment.Center) yang berfungsi untuk memastikan seluruh unsur antarmuka nantinya berada persis di bagian tengah layar.

Logika untuk menyimpan status dan proses pengacakan berada di dalam fungsi DiceWithButtonAndImages yang dimulai pada baris ke-62. Penyimpanan nilai angka dadu dikelola pada baris ke-63 dengan perintah var result1 by remember { mutableIntStateOf(0) } dan baris ke-64 dengan perintah var result2 by remember { mutableIntStateOf(0) }. Penggunaan fungsi remember dan mutableIntStateOf bertugas menyimpan status angka tersebut ke dalam memori dan secara otomatis memperbarui tampilan visual setiap kali nilai angka mengalami perubahan. Di samping itu, pada baris ke-66 terdapat deklarasi val context = LocalContext.current yang berfungsi mengambil informasi status aplikasi saat ini, di mana data ini menjadi syarat wajib untuk menampilkan pesan peringatan atau Toast.

Proses penentuan gambar dadu yang akan ditampilkan dikendalikan oleh struktur percabangan. Pada baris ke-68, kode val imageResource1 = when (result1) digunakan untuk mengevaluasi nilai dari dadu pertama. Blok kode ini mencocokkan angka hasil acakan dengan gambar yang sesuai, contohnya 1 -> R.drawable.dice_1 pada baris ke-70. Jika nilainya di luar jarak awal, maka kondisi else -> R.drawable.dice_6 pada baris ke-75 akan dieksekusi sebagai gambar bawaan. Pola pengecekan yang sama juga diterapkan untuk dadu kedua pada baris ke-78 melalui kode val imageResource2 = when (result2), yang akan memilih referensi gambar dari baris ke-79 hingga ke-85 tergantung pada nilai akhir variabel dadu kedua.

Penyusunan unsur visual antarmuka dilakukan mulai baris ke-88 menggunakan komponen Column, yang mengatur agar unsur tersusun secara vertikal dari atas ke bawah. Pengaturan horizontalAlignment = Alignment.CenterHorizontally pada baris ke-90 memastikan susunan tersebut rata tengah. Di dalam Column tersebut, terdapat komponen Row pada baris ke-92 yang berfungsi menyusun gambar dadu secara berdampingan. Gambar dadu pertama berada pada baris ke-96 dengan perintah Image(painter = painterResource(id = imageResource1), contentDescription = result1.toString()). Sebuah ruang pemisah selebar 16 piksel ditambahkan pada baris ke-101 dengan kode Spacer(modifier = Modifier.width(16.dp)), yang kemudian diikuti oleh gambar dadu kedua pada baris ke-103

menggunakan perintah Image(painter = painterResource(id = imageResource2), contentDescription = result2.toString()).

Bagian terakhir dari kode berfungsi untuk proses pengacakan angka melalui hubungan tombol. Jarak vertikal antara gambar dadu dan tombol diberikan melalui perintah Spacer(modifier = Modifier.height(32.dp)) pada baris ke-109. Komponen interaktif berupa tombol dibuat pada baris ke-111 dengan kode Button(onClick = {. Di dalam blok tersebut, nilai angka dadu yang baru dihasilkan secara acak melalui baris ke-112 dengan perintah result1 = (1..6).random() dan baris ke-113 dengan perintah result2 = (1..6).random(). Setelah nilai diperbarui, pengecekan angka dilakukan pada baris ke-115 dengan kondisi if (result1 == result2). Jika angkanya sama, baris ke-116 akan mengeksekusi perintah Toast.makeText(context, "Selamat, anda dapat dadu double", Toast.LENGTH_SHORT).show() untuk memunculkan teks notifikasi. Jika angkanya berbeda, blok kode akan mengeksekusi perintah Toast.makeText(context, "Anda belum beruntung!", Toast.LENGTH_SHORT).show() pada baris ke-118. Tombol pengacak ini diberi label visual berupa teks melalui perintah Text(text = "Roll") pada baris ke-121.

SOAL 2

1. Setelah melakukan riset pada buku ataupun jurnal mengenai paradigma imperatif dan deklaratif jadi penerapan pada UI :
    - Imperatif
        - Penerapan pada UI yaitu pada pengembangan aplikasi android, paradigma ini dilakukan dengan mengedit ui melalui XML secara manual berdasarkan perubahan data menggunakan bahasa seperti kotlin atau java.
        - Keunggulan paradigma ini stabilitas dalam sekuensial dan efisiensi memori awal, namun menghasilkan kode yang kompleks dan rawan kesalahan.
    - Deklaratif
        - Penerapan pada UI yaitu pada membuat franework modern seperti Jetpack Compose, developer cukup menjelaskan tampilan UI , lalu sistem akan menyesuaikan sesuai dengan data yang berjalan.
        - Keunggulan abstraksi lebih tinggi, logika yang lebih baik, dan pengembangan jangka panjang, namun latensi beban tinggi.

    Berdasarkan jurnal yang sudah didapat opini saya mengenai paradigma yang lebih prefer untuk dipilih iyalah deklaratif alasannnya :

    1. Beban kognitif yang berkurang, meningkatkan pemahaman dan penulisan kode pada logika bisnis dengan membebaskan pengembang dari beban teknis *low-level*.
    2. Efisiensi dalam pengembangan, meskipun ada kurangnya yaitu beban latensi yang berat, namun ada keuntungan untuk jangka panjang dan kode modular dan ringkas.
    3. Paradigma deklaratif lebih sesuai dengan kebutuhan industri saat ini.

SOAL 3

2. Berdasarkan jurnal dan artikel pada Java dan Kotlin sama sama menggunakan pilar OOP, tapi ada beberapa perbedaan mengenai OOP pada java dan OOP pada kotlin yaitu:
    1. Pada Java, variabel objek bisa bernilai null secara default, sehingga kesalahan seperti NullPointerException biasanya baru diketahui saat program dijalankan(Morelli & Walde, 2017). Pada Kotlin, ada fitur Null Safety yang berfungi menentukan variabel boleh null atau tidak ketika penulisan kode agar program aman (Riki Afriansyah, 2023). Ini berfungsi agar mengurangi resiko kegagalan aplikasi
    2. Pada OOP java kode lebih panjang dan harus ditulis lengkap tidak boleh tetinggal. Pada Kotlin OOP lebih ringkas dan lebih efisien dan tidak wajib menggunakan titik koma. Bisa berfokus pada logika bisnis dari pada ribetnya sintaks
    3. Java penulisannya lebih kompleks misalnya saat input data membutuhkan kelas tambahan misalnya scanner, pada Kotlin ada fitur bawaan lebih sederhana misalnya redline sehingga lebih mudah.
    4. Kotlin dibuat untuk melengkapi bukan untuk menggantikan Java. Keduanya saling berhubungan dan bisa digunakan secara bersamaan

Daftar Pustaka

Maraee, A., & Sturm, A. (2021). Imperative versus declarative constraint specification languages: a controlled experiment. *Software and Systems Modeling*, *20*(1), 27–48. https://doi.org/10.1007/s10270-020-00796-4

Morelli, R., & Walde, R. (2017). *Object-Oriented Problem Solving Third Edition*.

Nurhadi, N., & Muslim, A. (n.d.). *Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method* (Vol. 05, Number 05).

Riki Afriansyah, R. I. S. Mellani. (2023). *PEMROGRAMAN BERORIENTASI OBJEK DENGAN KOTLIN*. 155.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/MuhammadNafisPutra/Praktikum-PemrogramanMobile.git